Reactor Engineering Engine: Capabilities, Inputs, and Worked Command Examples

Reactor Engine User Guide

May 2, 2026

Contents

1 Purpose of the Code **2**

2 Before Running **2**
2.1 Advanced Libraries Used by the Project . . . 2

3 Modes Available **3**

4 Mode 1: Single-Reaction CSTR **3**
4.1 What the Code Computes . . . 3
4.2 Minimum Inputs . . . 4
4.3 Example 1A: Find CSTR Volume for a Target Conversion . . . 4
4.4 Example 1B: Find CSTR Conversion for a Known Volume . . . 5

5 Mode 2: Isothermal PFR **5**
5.1 What the Code Computes . . . 5
5.2 Minimum Inputs . . . 5
5.3 Example 2A: Liquid-Phase PFR Conversion . . . 6
5.4 Example 2B: PFR with a Different Temperature . . . 6

6 Mode 3: Gas-Phase PFR with Expansion **6**
6.1 What the Code Computes . . . 6
6.2 Minimum Inputs . . . 6
6.3 Example 3A: Gas Expansion Reduces Concentration . . . 7
6.4 Example 3B: Stronger Gas Expansion . . . 7

7 Mode 4: Non-Isothermal PFR **7**
7.1 What the Code Computes . . . 7
7.2 Minimum Inputs . . . 7
7.3 Example 4A: Exothermic PFR with Cooling . . . 8
7.4 Example 4B: Weak Cooling . . . 8
7.5 Example 4C: Gas-Phase Non-Isothermal PFR with Temperature Correction . . . 9

8 Mode 5: Series Batch Reactor, $A \to B \to C$ **9**
8.1 What the Code Computes . . . 9
8.2 Minimum Inputs . . . 9
8.3 Example 5A: Intermediate Product in Batch Reactor . . . 10
8.4 Example 5B: Faster Desired Step, Slower Undesired Step . . . 10
8.5 Example 5C: Undesired Step Is Fast . . . 10

9 Mode 6: Series CSTR, $A \to B \to C$ **11**
9.1 What the Code Computes . . . 11
9.2 Example 6A: CSTR Intermediate Optimization . . . 11
9.3 Example 6B: Larger Initial Concentration . . . 11
9.4 Example 6C: Different Rate Constants . . . 12

10 Mode 7: Power-Law Kinetic Fitting **12**
10.1 What the Code Computes . . . 12
10.2 Minimum Inputs . . . 12
10.3 Data Format A: Rate Already Given . . . 13
10.4 Example 7A: Fit a New Catalytic Rate Law . . . 13
10.5 Data Format B: Product Mole Fraction Given . . . 13
10.6 Example 7B: Gas-Phase Catalytic CSTR Fit . . . 14
10.7 Example 7C: Same Fit with a Plot . . . 14
10.8 Example 7D: Saving the Fit Plot . . . 14

11 Mode 8: Parallel-Reaction CSTR Optimization **14**
11.1 What the Code Computes . . . 14
11.2 Minimum Inputs . . . 15
11.3 Example 8A: Parallel CSTR with Selectivity and Profit . . . 15
11.4 Example 8B: Same Problem with a Plot . . . 16
11.5 Example 8C: Change the Undesired Reaction Order . . . 17
11.6 Example 8D: Higher Desired Product Price . . . 17

12 Mode 9: Species Properties with `thermo` or `CoolProp` **18**
12.1 What the Code Computes . . . 18
12.2 Minimum Inputs . . . 18
12.3 Example 9A: Water Properties with Automatic Backend . . . 18
12.4 Example 9B: Ethanol Properties Forced Through `thermo` . . . 18
12.5 Example 9C: Water Properties Forced Through `CoolProp` . . . 19

13 Mode 10: Species-Based Non-Isothermal PFR **19**
13.1 What the Code Computes . . . 19
13.2 Minimum Inputs . . . 19
13.3 Example 10A: Ethanol Oxidation-Type PFR with Real Heat Capacities . . . 20
13.4 Example 10B: Same PFR with an Inert Diluent . . . 20
13.5 Example 10C: Show the Conversion and Temperature Plots . . . 20

14 Common Input Checklist **21**
14.1 For Reactor Design Problems 21
14.2 For Kinetic Fitting Problems 21
14.3 For Parallel Selectivity Problems 22
14.4 For Real Species Property Problems 22

15 All Major Command-Line Inputs **23**

16 Summary **23**

1 Purpose of the Code

The main file is:

```
reactor_engine.py
```

It is a Python command-line tool for common chemical reaction engineering calculations. The code can compute reactor design, conversion profiles, temperature profiles, reaction selectivity, reaction yield, kinetic parameters from experimental data, parallel-reaction CSTR optimization, pure-species property lookups, and non-isothermal PFR calculations using real heat capacities from property libraries.

The general command pattern is:

```
python3 reactor_engine.py --mode MODE_NAME [inputs]
```

The most important input is always `--mode`, because it tells the code what type of problem to solve.

Copy-paste note. Long commands are written with backslashes. Copy the whole command block at once. Each line that ends in \ tells Terminal that the command continues on the next line. Do not copy a line that has only `--`; every option must stay together, such as `--parallel-ku2`.

2 Before Running

From Terminal, clone the project and go to the project folder:

```
git clone https://github.com/77bradbf/reactor-engine.git
cd reactor-engine
```

Install the base Python packages if needed:

```
python3 -m pip install -r requirements.txt
```

For advanced real-data/property calculations, install:

```
python3 -m pip install -r requirements-advanced.txt
```

2.1 Advanced Libraries Used by the Project

The advanced requirements file includes several scientific libraries. The current code uses some directly and keeps others available for future extensions:

Library	How It Helps
`pandas`	Reads CSV and Excel tables for kinetic fitting when available.
`openpyxl`	Lets `pandas` read `.xlsx` Excel files.
`thermo`	Provides chemical properties and molar heat capacities for real species.
`CoolProp`	Provides thermophysical properties for many common fluids.
`pint`	Installed for future unit-aware calculations.
`lmfit`	Installed for future nonlinear kinetic fitting and uncertainty estimates.
`cantera`	Installed for future detailed kinetics, thermodynamics, and reactor networks.

Show all available options:

```
python3 reactor_engine.py --help
```

Run without plots:

```
python3 reactor_engine.py --mode series-batch --no-show
```

Run with plots:

```
python3 reactor_engine.py --mode series-batch
```

Save plots instead of opening them:

```
python3 reactor_engine.py --mode series-batch --save-plots plots --no-show
```

3 Modes Available

Mode	What It Computes
`cstr`	Single-reaction CSTR conversion or required volume.
`pfr`	Single-reaction isothermal PFR conversion profile.
`nonisothermal-pfr`	Coupled conversion and temperature profile for a PFR.
`series-batch`	Batch reactor for first-order series reactions $A \to B \to C$.
`series-cstr`	CSTR for first-order series reactions $A \to B \to C$.
`kinetic-fit-power-law`	Fits $r = kx_1^\alpha x_2^\beta$ from experimental data.
`parallel-cstr`	Optimizes selectivity and profit for parallel CSTR reactions.
`species-properties`	Looks up real pure-species properties with `thermo` or `CoolProp`.
`species-nonisothermal-pfr`	Solves a non-isothermal PFR using real species heat capacities from `thermo` or `CoolProp`.

4 Mode 1: Single-Reaction CSTR

4.1 What the Code Computes

For a CSTR, the code uses the design equation:

$$V = \frac{F_{A0}X}{-r_A} \tag{1}$$

The default reaction rate is second-order:

$$-r_A = kC_AC_B \tag{2}$$

The default Arrhenius equation is:

$$k = A\exp\left(-\frac{E_a}{RT}\right) \tag{3}$$

with:

$$A = 5.7 \times 10^5$$
$$E_a = 33400 \text{ J/mol}$$
$$R = 8.314 \text{ J/(mol K)}$$

4.2 Minimum Inputs

For a CSTR, you must know either:

- target conversion X, to calculate volume, or
- reactor volume V, to calculate conversion.

Useful inputs are:

- `--target-x`
- `--volume`
- `--temperature`
- `--fa0`
- `--fb0`
- `--v0`

4.3 Example 1A: Find CSTR Volume for a Target Conversion

Problem statement. A liquid-phase reaction follows:

$$-r_A = kC_AC_B \tag{4}$$

A feed contains $F_{A0} = 10$ mol/min and $F_{B0} = 15$ mol/min. The inlet volumetric flow rate is $v_0 = 100$ L/min. The reactor is isothermal at 360 K. Find the CSTR volume required for 65% conversion.

Command.

```
python3 reactor_engine.py \
  --mode cstr \
  --target-x 0.65 \
  --temperature 360 \
  --fa0 10 \
  --fb0 15 \
  --v0 100 \
  --no-show
```

What each input means.

Input	Meaning
`--mode cstr`	Use CSTR design equation.
`--target-x 0.65`	Desired conversion is 65%.
`--temperature 360`	Reactor temperature is 360 K.
`--fa0 10`	Inlet molar flow of A.
`--fb0 15`	Inlet molar flow of B.
`--v0 100`	Inlet volumetric flow rate.
`--no-show`	Print output without opening a plot window.

4.4 Example 1B: Find CSTR Conversion for a Known Volume

Problem statement. The same reaction is run in a 250 L CSTR at 360 K. Find the exit conversion.

Command.

```
python3 reactor_engine.py \
  --mode cstr \
  --volume 250 \
  --temperature 360 \
  --fa0 10 \
  --fb0 15 \
  --v0 100 \
  --no-show
```

5 Mode 2: Isothermal PFR

5.1 What the Code Computes

For an isothermal plug flow reactor, the code integrates:

$$\frac{dX}{dV} = \frac{-r_A}{F_{A0}} \tag{5}$$

The default rate law is:

$$-r_A = kC_AC_B \tag{6}$$

5.2 Minimum Inputs

At minimum, use:

- `--mode pfr`
- `--volume`

Add feed and temperature data when the problem gives them.

5.3 Example 2A: Liquid-Phase PFR Conversion

Problem statement. A second-order liquid-phase reaction is carried out in a 400 L PFR at 375 K. The feed contains $F_{A0} = 8$ mol/min, $F_{B0} = 12$ mol/min, and $v_0 = 80$ L/min. Find the exit conversion.

Command.

```
python3 reactor_engine.py \
  --mode pfr \
  --volume 400 \
  --temperature 375 \
  --fa0 8 \
  --fb0 12 \
  --v0 80 \
  --no-show
```

5.4 Example 2B: PFR with a Different Temperature

Problem statement. Repeat the PFR calculation at 390 K to see the effect of temperature on conversion.

Command.

```
python3 reactor_engine.py \
  --mode pfr \
  --volume 400 \
  --temperature 390 \
  --fa0 8 \
  --fb0 12 \
  --v0 80 \
  --no-show
```

6 Mode 3: Gas-Phase PFR with Expansion

6.1 What the Code Computes

For gas-phase reactions, volumetric flow may change with conversion. The code can use:

$$v = v_0(1 + \epsilon X) \tag{7}$$

where:

$$\epsilon = y_{A0}\delta \tag{8}$$

6.2 Minimum Inputs

Use the normal PFR inputs plus:

- `--gas-phase`
- `--y-a0`
- `--delta`

6.3 Example 3A: Gas Expansion Reduces Concentration

Problem statement. A gas-phase reaction is carried out in a 400 L PFR at 375 K. The inlet mole fraction of A is $y_{A0} = 0.4$ and $\delta = 1.5$. Find the conversion when expansion is included.

Command.

```
python3 reactor_engine.py \
  --mode pfr \
  --volume 400 \
  --temperature 375 \
  --fa0 8 \
  --fb0 12 \
  --v0 80 \
  --gas-phase \
  --y-a0 0.4 \
  --delta 1.5 \
  --no-show
```

6.4 Example 3B: Stronger Gas Expansion

Problem statement. Repeat the gas-phase PFR problem with $\delta = 2.5$.

Command.

```
python3 reactor_engine.py \
  --mode pfr \
  --volume 400 \
  --temperature 375 \
  --fa0 8 \
  --fb0 12 \
  --v0 80 \
  --gas-phase \
  --y-a0 0.4 \
  --delta 2.5 \
  --no-show
```

7 Mode 4: Non-Isothermal PFR

7.1 What the Code Computes

The code solves the coupled PFR material and energy balance:

$$\frac{dX}{dV} = \frac{-r_A}{F_{A0}} \tag{9}$$

$$\frac{dT}{dV} = \frac{Ua(T_a - T) + (-r_A)(-\Delta H_{rxn})}{\sum F_i C_{p,i}} \tag{10}$$

7.2 Minimum Inputs

Use:

- `--mode nonisothermal-pfr`

- `--volume`
- `--t0`
- `--delta-h`
- `--ua`
- `--ta`
- heat capacities if different from defaults

7.3 Example 4A: Exothermic PFR with Cooling

Problem statement. An exothermic liquid-phase reaction is carried out in a 300 L PFR. Feed enters at 340 K with $F_{A0} = 12$ mol/min, $F_{B0} = 18$ mol/min, and $v_0 = 120$ L/min. The heat of reaction is -45000 J/mol, $Ua = 350$, and coolant temperature is 320 K. Find conversion and exit temperature.

Command.

```
python3 reactor_engine.py \
  --mode nonisothermal-pfr \
  --volume 300 \
  --fa0 12 \
  --fb0 18 \
  --v0 120 \
  --t0 340 \
  --delta-h -45000 \
  --ua 350 \
  --ta 320 \
  --cp-a 95 \
  --cp-b 110 \
  --cp-p 120 \
  --no-show
```

7.4 Example 4B: Weak Cooling

Problem statement. Repeat the previous problem with weaker heat removal, $Ua = 100$.

Command.

```
python3 reactor_engine.py \
  --mode nonisothermal-pfr \
  --volume 300 \
  --fa0 12 \
  --fb0 18 \
  --v0 120 \
  --t0 340 \
  --delta-h -45000 \
  --ua 100 \
  --ta 320 \
  --cp-a 95 \
  --cp-b 110 \
  --cp-p 120 \
  --no-show
```

7.5 Example 4C: Gas-Phase Non-Isothermal PFR with Temperature Correction

Problem statement. A gas-phase non-isothermal PFR has both conversion expansion and temperature expansion. Use $v = v_0(1 + \epsilon X)(T/T_0)$.

Command.

```
python3 reactor_engine.py \
  --mode nonisothermal-pfr \
  --volume 250 \
  --fa0 10 \
  --fb0 15 \
  --v0 100 \
  --t0 350 \
  --gas-phase \
  --y-a0 0.5 \
  --delta 1.0 \
  --ideal-gas-temperature-correction \
  --delta-h -30000 \
  --ua 250 \
  --ta 330 \
  --no-show
```

8 Mode 5: Series Batch Reactor, $A \to B \to C$

8.1 What the Code Computes

For first-order series reactions:

$$A \xrightarrow{k_1} B \xrightarrow{k_2} C \tag{11}$$

The batch reactor expressions are:

$$C_A(t) = C_{A0} e^{-k_1 t} \tag{12}$$

$$C_B(t) = \frac{C_{A0} k_1}{k_2 - k_1} \left(e^{-k_1 t} - e^{-k_2 t} \right) \tag{13}$$

$$C_C(t) = C_{A0} - C_A(t) - C_B(t) \tag{14}$$

The time for maximum B is:

$$t_{max} = \frac{\ln(k_1/k_2)}{k_1 - k_2} \tag{15}$$

8.2 Minimum Inputs

Use:

- `--mode series-batch`
- `--series-k1`
- `--series-k2`
- `--ca0`

8.3 Example 5A: Intermediate Product in Batch Reactor

Problem statement. The elementary liquid-phase series reaction $A \to B \to C$ is carried out in a batch reactor. Let $k_1 = 0.5$ h^{-1}, $k_2 = 0.2$ h^{-1}, and $C_{A0} = 2$ mol/L. Plot concentrations from 0 to 12 h and find the time for maximum B.

Command.

```
python3 reactor_engine.py \
  --mode series-batch \
  --series-k1 0.5 \
  --series-k2 0.2 \
  --ca0 2 \
  --time-start 0 \
  --time-end 12
```

Use `--no-show` if you do not want the plot:

```
python3 reactor_engine.py \
  --mode series-batch \
  --series-k1 0.5 \
  --series-k2 0.2 \
  --ca0 2 \
  --time-start 0 \
  --time-end 12 \
  --no-show
```

8.4 Example 5B: Faster Desired Step, Slower Undesired Step

Problem statement. A batch reactor has $k_1 = 0.8$ h^{-1} and $k_2 = 0.15$ h^{-1}. The initial A concentration is 3 mol/L. Find the best time to stop the batch to maximize B.

Command.

```
python3 reactor_engine.py \
  --mode series-batch \
  --series-k1 0.8 \
  --series-k2 0.15 \
  --ca0 3 \
  --time-start 0 \
  --time-end 20 \
  --no-show
```

8.5 Example 5C: Undesired Step Is Fast

Problem statement. A batch reactor has $k_1 = 0.4$ h^{-1} and $k_2 = 1.1$ h^{-1}. Since B disappears quickly, find the maximum possible concentration of B.

Command.

```
python3 reactor_engine.py \
  --mode series-batch \
  --series-k1 0.4 \
  --series-k2 1.1 \
  --ca0 2.5 \
  --time-start 0 \
```

```
  --time-end 10 \
  --no-show
```

9 Mode 6: Series CSTR, $A \to B \to C$

9.1 What the Code Computes

For a CSTR with first-order series reactions:

$$C_A(\tau) = \frac{C_{A0}}{1 + k_1\tau} \tag{16}$$

$$C_B(\tau) = \frac{k_1\tau C_A(\tau)}{1 + k_2\tau} \tag{17}$$

$$C_C(\tau) = C_{A0} - C_A(\tau) - C_B(\tau) \tag{18}$$

The residence time that maximizes B is:

$$\tau_{opt} = \frac{1}{\sqrt{k_1 k_2}} \tag{19}$$

9.2 Example 6A: CSTR Intermediate Optimization

Problem statement. The same reaction $A \to B \to C$ is carried out in a CSTR. Use $k_1 = 0.5$ h^{-1}, $k_2 = 0.2$ h^{-1}, and $C_{A0} = 2$ mol/L. Find the residence time that maximizes B.

Command.

```
python3 reactor_engine.py \
  --mode series-cstr \
  --series-k1 0.5 \
  --series-k2 0.2 \
  --ca0 2 \
  --time-start 0 \
  --time-end 12 \
  --no-show
```

9.3 Example 6B: Larger Initial Concentration

Problem statement. A CSTR uses the same rate constants but $C_{A0} = 5$ mol/L. Find the maximum B concentration and the optimum residence time.

Command.

```
python3 reactor_engine.py \
  --mode series-cstr \
  --series-k1 0.5 \
  --series-k2 0.2 \
  --ca0 5 \
  --time-start 0 \
  --time-end 12 \
  --no-show
```

9.4 Example 6C: Different Rate Constants

Problem statement. A CSTR has $k_1 = 1.2\ \mathrm{h}^{-1}$ and $k_2 = 0.3\ \mathrm{h}^{-1}$. The feed concentration of A is 1.8 mol/L. Find the residence time that maximizes B.

Command.

```
python3 reactor_engine.py \
  --mode series-cstr \
  --series-k1 1.2 \
  --series-k2 0.3 \
  --ca0 1.8 \
  --time-start 0 \
  --time-end 10 \
  --no-show
```

10 Mode 7: Power-Law Kinetic Fitting

10.1 What the Code Computes

This mode fits a two-reactant power-law rate model:

$$r = k x_1^{\alpha} x_2^{\beta} \tag{20}$$

Taking logs gives:

$$\ln r = \ln k + \alpha \ln x_1 + \beta \ln x_2 \tag{21}$$

The code uses linear least squares on the log-transformed data to calculate:

- α
- β
- k
- R^2 on $\ln(r)$
- predicted rates
- an observed-versus-predicted plot

10.2 Minimum Inputs

If using the built-in photo example, only this is needed:

```
python3 reactor_engine.py --mode kinetic-fit-power-law --no-show
```

For new data, use:

- `--fit-data-csv examples/myfile.csv`, or
- `--fit-data-csv examples/myfile.xlsx`

The option is called `--fit-data-csv`, but it accepts both CSV and Excel files.

10.3 Data Format A: Rate Already Given

Use this table if the problem directly gives the measured rate:

x1	x2	rate
0.20	0.30	0.0048
0.40	0.30	0.0096
0.20	0.60	0.0068
0.50	0.50	0.0141
0.80	0.40	0.0202

Save this as:

```
examples/power_fit_example.xlsx
```

or:

```
examples/power_fit_example.csv
```

Then run:

```
python3 reactor_engine.py \
  --mode kinetic-fit-power-law \
  --fit-data-csv examples/power_fit_example.xlsx \
  --no-show
```

10.4 Example 7A: Fit a New Catalytic Rate Law

Problem statement. A catalytic reaction $A + B \rightarrow P$ follows:

$$r_P = kP_A^{\alpha} P_B^{\beta} \tag{22}$$

The experimental data are shown in the previous table. Determine α, β, and k.

Command.

```
python3 reactor_engine.py \
  --mode kinetic-fit-power-law \
  --fit-data-csv examples/power_fit_example.xlsx \
  --no-show
```

10.5 Data Format B: Product Mole Fraction Given

Use this table if the problem gives total outlet flow and product mole fraction:

total_flow	x1	x2	y_product
0.61	0.50	0.51	0.07
0.60	1.00	0.54	0.16
0.30	0.40	0.60	0.16
0.72	0.60	0.60	0.10
0.52	0.60	0.40	0.06

The code calculates:

$$r = \frac{F_T y_P}{W_{cat}} \tag{23}$$

where F_T is total molar flow, y_P is product mole fraction, and W_{cat} is catalyst mass.

10.6 Example 7B: Gas-Phase Catalytic CSTR Fit

Problem statement. The irreversible gas-phase reaction

$$H_2 + C_2H_6 \rightarrow 2CH_4 \tag{24}$$

is carried out over nickel catalyst. The model is:

$$r'_{CH_4} = kP^{\alpha}_{C_2H_6}P^{\beta}_{H_2} \tag{25}$$

A CSTR contains four baskets with 10 g catalyst each, so $W_{cat} = 40$ g. Determine α, β, and k.
Command.

```
python3 reactor_engine.py \
  --mode kinetic-fit-power-law \
  --fit-data-csv examples/kinetic_fit_photo.csv \
  --catalyst-weight 40 \
  --no-show
```

10.7 Example 7C: Same Fit with a Plot

Remove `--no-show`:

```
python3 reactor_engine.py \
  --mode kinetic-fit-power-law \
  --fit-data-csv examples/kinetic_fit_photo.csv \
  --catalyst-weight 40
```

10.8 Example 7D: Saving the Fit Plot

```
python3 reactor_engine.py \
  --mode kinetic-fit-power-law \
  --fit-data-csv examples/kinetic_fit_photo.csv \
  --catalyst-weight 40 \
  --save-plots plots \
  --no-show
```

11 Mode 8: Parallel-Reaction CSTR Optimization

11.1 What the Code Computes

This mode handles parallel reactions:

$$A + B \rightarrow D \qquad \text{desired product} \tag{26}$$
$$A + B \rightarrow U_1 \qquad \text{undesired product} \tag{27}$$
$$A + B \rightarrow U_2 \qquad \text{undesired product} \tag{28}$$

The general rate laws are:

$$r_D = k_D C_A^{a_D} C_B^{b_D} \tag{29}$$
$$r_{U1} = k_{U1} C_A^{a_{U1}} C_B^{b_{U1}} \tag{30}$$
$$r_{U2} = k_{U2} C_A^{a_{U2}} C_B^{b_{U2}} \tag{31}$$

The instantaneous selectivity is:

$$S_{D/(U1+U2)} = \frac{r_D}{r_{U1} + r_{U2}} \tag{32}$$

The code finds the exit C_A that maximizes selectivity. It also estimates residence time, volumetric flow rate, and profit difference using old and optimized outlet concentrations.

11.2 Minimum Inputs

Important inputs are:

- feed concentrations: `--parallel-ca0`, `--parallel-cb0`
- known exit B concentration: `--parallel-exit-cb`
- reactor volume: `--parallel-volume`
- rate constants: `--parallel-kd`, `--parallel-ku1`, `--parallel-ku2`
- reaction orders: `--d-exp-a`, `--d-exp-b`, etc.
- prices and costs if profit is required

11.3 Example 8A: Parallel CSTR with Selectivity and Profit

Problem statement. A 500 L CSTR is used for the liquid-phase parallel reactions:

$$A + B \rightarrow D$$
$$A + B \rightarrow U_1$$
$$A + B \rightarrow U_2$$

The feed has $C_{A0} = 2.5$ mol/L and $C_{B0} = 1.8$ mol/L. The outlet concentration of B is 0.6 mol/L. The rate laws are:

$$r_D = 0.45 C_A^1 C_B^1$$
$$r_{U1} = 0.12 C_A^{0.5} C_B^1$$
$$r_{U2} = 0.08 C_A^2 C_B^1$$

Product D sells for 75 dollars/mol, and disposal of each undesired product costs 15 dollars/mol. The previous outlet stream had $D = 1.1$, $U_1 = 0.4$, $U_2 = 0.5$ mol/L. The optimized outlet stream has $D = 1.4$, $U_1 = 0.25$, $U_2 = 0.35$ mol/L. Find the exit C_A that maximizes selectivity, the residence time, volumetric flow rate, and profit increase.

Command.

```
python3 reactor_engine.py \
  --mode parallel-cstr \
  --parallel-ca0 2.5 \
  --parallel-cb0 1.8 \
  --parallel-exit-cb 0.6 \
  --parallel-volume 500 \
  --parallel-kd 0.45 \
  --parallel-ku1 0.12 \
  --parallel-ku2 0.08 \
  --d-exp-a 1 \
  --d-exp-b 1 \
  --u1-exp-a 0.5 \
  --u1-exp-b 1 \
  --u2-exp-a 2 \
  --u2-exp-b 1 \
  --desired-price 75 \
  --undesired-cost 15 \
  --previous-d 1.1 \
  --previous-u1 0.4 \
  --previous-u2 0.5 \
  --optimized-d 1.4 \
  --optimized-u1 0.25 \
  --optimized-u2 0.35 \
  --no-show
```

11.4 Example 8B: Same Problem with a Plot

Remove `--no-show`:

```
python3 reactor_engine.py \
  --mode parallel-cstr \
  --parallel-ca0 2.5 \
  --parallel-cb0 1.8 \
  --parallel-exit-cb 0.6 \
  --parallel-volume 500 \
  --parallel-kd 0.45 \
  --parallel-ku1 0.12 \
  --parallel-ku2 0.08 \
  --d-exp-a 1 \
  --d-exp-b 1 \
  --u1-exp-a 0.5 \
  --u1-exp-b 1 \
  --u2-exp-a 2 \
  --u2-exp-b 1 \
  --desired-price 75 \
  --undesired-cost 15 \
  --previous-d 1.1 \
```

```
  --previous-u1 0.4 \
  --previous-u2 0.5 \
  --optimized-d 1.4 \
  --optimized-u1 0.25 \
  --optimized-u2 0.35
```

11.5 Example 8C: Change the Undesired Reaction Order

Problem statement. Suppose the second undesired reaction is less sensitive to A:

$$r_{U2} = 0.08 C_A^{1.5} C_B \tag{33}$$

Find the new optimum.

Command.

```
python3 reactor_engine.py \
  --mode parallel-cstr \
  --parallel-ca0 2.5 \
  --parallel-cb0 1.8 \
  --parallel-exit-cb 0.6 \
  --parallel-volume 500 \
  --parallel-kd 0.45 \
  --parallel-ku1 0.12 \
  --parallel-ku2 0.08 \
  --d-exp-a 1 \
  --d-exp-b 1 \
  --u1-exp-a 0.5 \
  --u1-exp-b 1 \
  --u2-exp-a 1.5 \
  --u2-exp-b 1 \
  --no-show
```

11.6 Example 8D: Higher Desired Product Price

Problem statement. If product D sells for 120 dollars/mol instead of 75 dollars/mol, calculate the new profit increase.

Command.

```
python3 reactor_engine.py \
  --mode parallel-cstr \
  --parallel-ca0 2.5 \
  --parallel-cb0 1.8 \
  --parallel-exit-cb 0.6 \
  --parallel-volume 500 \
  --parallel-kd 0.45 \
  --parallel-ku1 0.12 \
  --parallel-ku2 0.08 \
  --d-exp-a 1 \
  --d-exp-b 1 \
  --u1-exp-a 0.5 \
  --u1-exp-b 1 \
  --u2-exp-a 2 \
  --u2-exp-b 1 \
```

```
  --desired-price 120 \
  --undesired-cost 15 \
  --previous-d 1.1 \
  --previous-u1 0.4 \
  --previous-u2 0.5 \
  --optimized-d 1.4 \
  --optimized-u1 0.25 \
  --optimized-u2 0.35 \
  --no-show
```

12 Mode 9: Species Properties with `thermo` or `CoolProp`

12.1 What the Code Computes

This mode looks up real pure-species properties at a specified temperature and pressure. It can use `thermo`, `CoolProp`, or `auto`. With `auto`, the code tries `thermo` first and then `CoolProp`.

Depending on the backend and species, the output can include molecular weight, phase, density, heat capacity, enthalpy, entropy, vapor pressure, viscosity, thermal conductivity, critical properties, and other available properties.

12.2 Minimum Inputs

- `--mode species-properties`
- `--species`: chemical name or backend-recognized species name
- `--temperature`: temperature in K
- `--pressure`: pressure in Pa
- `--property-backend`: `auto`, `thermo`, or `coolprop`

12.3 Example 9A: Water Properties with Automatic Backend

Problem statement. Find real pure-water properties at 350 K and 1 atm. Let the code choose the backend automatically.

Command.

```
python3 reactor_engine.py \
  --mode species-properties \
  --species water \
  --temperature 350 \
  --pressure 101325 \
  --property-backend auto \
  --no-show
```

12.4 Example 9B: Ethanol Properties Forced Through `thermo`

Problem statement. Find ethanol properties at 330 K and 1 atm using the `thermo` library.

Command.

```
python3 reactor_engine.py \
  --mode species-properties \
  --species ethanol \
  --temperature 330 \
  --pressure 101325 \
  --property-backend thermo \
  --no-show
```

12.5 Example 9C: Water Properties Forced Through `CoolProp`

Problem statement. Find water properties at 350 K and 1 atm using `CoolProp`. Some `CoolProp` names use capital letters, such as `Water`.

Command.

```
python3 reactor_engine.py \
  --mode species-properties \
  --species Water \
  --temperature 350 \
  --pressure 101325 \
  --property-backend coolprop \
  --no-show
```

13 Mode 10: Species-Based Non-Isothermal PFR

13.1 What the Code Computes

This mode solves the same coupled PFR design and energy balance as `nonisothermal-pfr`, but it calculates the heat-capacity flow using real species molar heat capacities from `thermo` or `CoolProp`:

$$\frac{dX}{dV} = \frac{-r_A}{F_{A0}} \tag{34}$$

$$\frac{dT}{dV} = \frac{Ua(T_a - T) + (-r_A)(-\Delta H_{rxn})}{\sum F_i C_{p,i}(T, P)} \tag{35}$$

This is more realistic than entering constant `--cp-a`, `--cp-b`, and `--cp-p` by hand.

13.2 Minimum Inputs

- reactor/feed inputs: `--volume`, `--fa0`, `--fb0`, `--v0`, `--t0`
- species names: `--species-a`, `--species-b`, `--species-p`
- heat inputs: `--delta-h`, `--ua`, `--ta`
- property inputs: `--pressure`, `--property-backend`
- optional inert: `--fi0` and `--species-i`

13.3 Example 10A: Ethanol Oxidation-Type PFR with Real Heat Capacities

Problem statement. A PFR is modeled with A = ethanol, B = oxygen, and P = acetaldehyde. The feed has $F_{A0} = 2$ mol/time and $F_{B0} = 3$ mol/time, with $v_0 = 50$ volume/time and $T_0 = 330$ K. The reactor volume is 50, $\Delta H_{rxn} = -50000$ J/mol A, $Ua = 120$, and $T_a = 310$ K. Compute conversion and exit temperature using real heat capacities from `thermo`.

Command.

```
python3 reactor_engine.py \
  --mode species-nonisothermal-pfr \
  --volume 50 \
  --fa0 2 \
  --fb0 3 \
  --v0 50 \
  --t0 330 \
  --species-a ethanol \
  --species-b oxygen \
  --species-p acetaldehyde \
  --delta-h -50000 \
  --ua 120 \
  --ta 310 \
  --pressure 101325 \
  --property-backend thermo \
  --no-show
```

13.4 Example 10B: Same PFR with an Inert Diluent

Problem statement. Repeat the previous reactor, but add 5 mol/time of nitrogen inert. Because an inert is present, include both `--fi0` and `--species-i`.

Command.

```
python3 reactor_engine.py \
  --mode species-nonisothermal-pfr \
  --volume 50 \
  --fa0 2 \
  --fb0 3 \
  --fi0 5 \
  --v0 100 \
  --t0 330 \
  --species-a ethanol \
  --species-b oxygen \
  --species-p acetaldehyde \
  --species-i nitrogen \
  --delta-h -50000 \
  --ua 120 \
  --ta 310 \
  --pressure 101325 \
  --property-backend thermo \
  --no-show
```

13.5 Example 10C: Show the Conversion and Temperature Plots

Remove `--no-show` to open the Matplotlib plots:

```
python3 reactor_engine.py \
  --mode species-nonisothermal-pfr \
  --volume 50 \
  --fa0 2 \
  --fb0 3 \
  --v0 50 \
  --t0 330 \
  --species-a ethanol \
  --species-b oxygen \
  --species-p acetaldehyde \
  --delta-h -50000 \
  --ua 120 \
  --ta 310 \
  --pressure 101325 \
  --property-backend thermo
```

14 Common Input Checklist

14.1 For Reactor Design Problems

Ask:

1. What reactor type is it: CSTR, PFR, batch, or non-isothermal PFR?
2. What is the rate law?
3. What are the rate constants and temperature?
4. What are the feed molar flows or concentrations?
5. Is the problem asking for conversion, volume, residence time, selectivity, yield, or temperature?
6. Is it gas phase? If yes, is expansion important?

14.2 For Kinetic Fitting Problems

Ask:

1. Does the problem give measured rates directly?
2. Or does it give flow rate and product mole fraction?
3. What are x_1 and x_2? Usually pressures or concentrations.
4. What is the catalyst weight, if rates must be calculated from product flow?
5. Are all values positive? Log fitting requires positive data.

14.3 For Parallel Selectivity Problems

Ask:

1. Which product is desired?
2. Which products are undesired?
3. What are the rate laws for each path?
4. Which concentration is fixed at the exit?
5. What concentration should be optimized?
6. Is profit required? If yes, what are product values and disposal costs?

14.4 For Real Species Property Problems

Ask:

1. Which species names should the property library use?
2. What are the temperature and pressure?
3. Should the backend be `auto`, `thermo`, or `coolprop`?
4. For species-based PFRs, which species correspond to A, B, product P, and optional inert I?
5. Is the heat of reaction exothermic or endothermic, and are the energy units consistent?

15 All Major Command-Line Inputs

Input	Meaning
`--mode`	Selects the calculation type.
`--volume`	Reactor volume.
`--target-x`	Target conversion for CSTR volume calculation.
`--temperature`	Isothermal reactor temperature.
`--fa0`, `--fb0`	Inlet molar flows of A and B.
`--v0`	Inlet volumetric flow rate.
`--arrhenius-a`, `--ea`	Arrhenius parameters.
`--gas-phase`	Turns on gas expansion.
`--y-a0`, `--delta`	Gas expansion parameters.
`--delta-h`, `--ua`, `--ta`	Energy balance parameters.
`--series-k1`, `--series-k2`	Series reaction rate constants.
`--ca0`	Initial or feed concentration of A for series reactions.
`--fit-data-csv`	CSV or Excel data file for kinetic fitting.
`--catalyst-weight`	Catalyst mass for product-flow data.
`--parallel-ca0`, `--parallel-cb0`	Feed concentrations for parallel CSTR.
`--parallel-kd`, `--parallel-ku1`, `--parallel-ku2`	Parallel reaction rate constants.
`--d-exp-a`, `--u1-exp-a`, etc.	Reaction orders in the parallel rate laws.
`--desired-price`, `--undesired-cost`	Economic values for profit calculation.
`--species`	Pure species name for property lookup.
`--species-a`, `--species-b`, `--species-p`	Species names for reactant A, reactant B, and product P.
`--species-i`	Optional inert species name when `--fi0` is nonzero.
`--pressure`	Pressure for real-property calculations, Pa.
`--property-backend`	Selects `auto`, `thermo`, or `coolprop`.
`--no-show`	Suppresses plot windows.
`--save-plots`	Saves plots to a folder.

16 Summary

The code is most useful when you first classify the problem:

- Use `cstr` for a single CSTR design equation.
- Use `pfr` for a single isothermal PFR profile.
- Use `nonisothermal-pfr` when temperature changes with volume.
- Use `series-batch` for batch $A \to B \to C$ problems.
- Use `series-cstr` for CSTR $A \to B \to C$ problems.

- Use `kinetic-fit-power-law` when the problem gives experimental data and asks for reaction orders and k.
- Use `parallel-cstr` when the problem asks for selectivity, yield, waste products, or profit optimization in parallel reactions.
- Use `species-properties` when the problem asks for real physical properties of a chemical species.
- Use `species-nonisothermal-pfr` when the energy balance should use real species heat capacities from property libraries.